

RAPPORT PPIL

Table des matières

Introduction :	3
UML :	3
Forme Géométrique :	3
Transformations Géométriques :	5
Dessin d'une forme :	5
Sauvegarder/Chargement sur un fichier disque :	6
Fonctionnalités diverses :	7
Documentation :	7

Introduction :

Le rapport ci présent est le résultat ainsi que la description des divers méthodes et conceptions utilisées durant l'intégralité du projet de PPIL afin de répondre aux divers critères dont doit se soucier n'importe quel développeur vis-à-vis de la conception d'un logiciel : la fonctionnalité, la robustesse, l'extensibilité et la simplicité de maintenance. Le rapport se déroulera en fonction de l'ordre des services qui ont été demandées de réaliser. Les classes présentées dans ce rapport auront une majuscule au début pour mieux les identifier.

UML :

Les diagrammes UML prenant trop de place dans ce document, nous avons décidé de ne pas les inclure ici, d'où le fait que ce rapport est légèrement plus court que prévu. Cependant, il est possible de les consulter respectivement dans les dossiers client/doc/uml et serveur/doc/uml, ceux-ci étant disponible sous différents formats.

Forme Géométrique :

La première chose qui nous est demandé est la réalisation des formes géométriques. Si cet exercice semble dans un premier temps simple, une très mauvaise conception des classes permettant de réaliser ses formes peuvent mener directement à de sérieux problèmes pour la suite de la réalisation du projet. Par conséquent, cette partie a été la plus longue à réaliser, non par le nombre de ligne à écrire ou la complexité des algorithmes à étudier, mais tout simplement dans la réflexion de la meilleure manière de réaliser ses classes. Il a été conclu pour cela de décomposer les formes de la manière suivante :

- Dans un premier temps, nous avons réalisé une énumération de Couleur, telle que celle-ci possède des objets statiques de type couleur nommées par l'ensemble des couleurs qui devaient être utilisées (« Black », « Blue », « Red », « Green », « Yellow », « Cyan »). Cela permet tout simplement d'avoir un champ symbolique pour nos classes à la place d'un string ou d'un entier, en plus de minimiser le nombre d'objet Couleur puisque ces symboles sont de fait statiques.
- Dans un second temps, nous avons réalisé une classe abstraite Forme telle que celle-ci possède une couleur. Cette classe permet dans un premier d'établir des fonctions qui sont communes à n'importe quelles formes, telle que toString(), getCouleur(), setCouleur(), getAire(), getPoints(), translation(), homothetie(), rotation(), transformationEcran(), ainsi que les opérateurs string() et <<. Elle permet donc de définir des fonctions que chaque classe doit implémenter lorsque qu'elle dérive de Forme (getPoints(), translation(), homothetie(), etc...), mais aussi

de définir des fonctions qui n'auront plus à être définies par classes enfants, et qui sont pourtant grandement utiles (`operator string()`, `<<`, etc...)

- Dans un troisième temps, nous avons conçu la classe abstraite `Forme Simple` qui caractérise l'ensemble des formes simples. Elle implémente certaines fonctions laissées comme purement abstraites par `Forme`, et définit de nouvelles fonctions abstraites qui sont communes à l'ensemble des `Formes Simples`, dont la première caractéristique est que celles-ci peuvent être décrites par leurs ensembles de points (qui sera encodées ici comme des vecteurs pour simplifier les calculs).
- Dans un quatrième temps, nous avons conçu la classe `Forme Composée`, qui est composée de plusieurs `Formes`. Elle implémente à son tour des fonctions de `Forme`, mais contrairement à `Forme Simple`, elle ne définit aucune fonction abstraite. Étant donné qu'une `Forme Composée` est composée de `Forme`, alors pour certaines opérations, il suffira de les traiter pour chaque `Forme` (l'homothétie d'une forme composée est l'homothétie de l'ensemble de ses `Formes Simples`). Cependant, cette classe doit répondre au critère qu'une `Forme` ne peut pas appartenir à plusieurs `Formes Composées`, il a été décidé que lors de l'ajout d'une `Forme`, qu'il soit décidé d'en ajouter en réalité sa copie et non elle-même. Cela implique par conséquent une gestion manuelle du constructeur par copie, du clone, de l'opérateur `=`, et de la destruction de la `Forme Composée`, chose qui a été faite. Cette implémentation est le résultat très malheureux des problèmes techniques du C++, puisqu'il était supposé dans un premier temps de ne pas ajouter la copie d'une `Forme`, mais la `Forme` elle-même. Cependant, cela pose un certain nombre de problèmes, puisque cela implique que la destruction d'une `Forme Composée` n'implique pas la destruction de l'ensemble de ses `Formes Simples`, mais dans le cas où nous ferions la copie d'une `Forme Composée`, alors il faut tout de même copier un à un l'ensemble de ses `Formes`, ce qui implique que la copie de ses `Formes` n'ont pas d'autres origines ou d'autres moyens d'accès que la `Forme Composée`, et que par conséquent, à cause de la logique précédente de la destruction d'une `Forme Composée`, il y aurait une fuite de mémoire à cause de ces copies. Même si ce problème aurait pu être corrigé entre autres grâce à l'implémentation des pointeurs intelligents, le manque de temps et d'énergie afin de les comprendre nous a poussé à ne pas les utiliser.
- Enfin, dans un dernier temps vient la réalisation des classes `Segment`, `Cercle`, `Triangle` et `Polygone`, qui implémente `Forme Simple`, et qui, hormis quelques opérations sur les points en eux-mêmes (qui sont finalement des vecteurs), et l'implémentation des fonctions laissées purement abstraites par `Forme`, n'apportent pas tant d'éléments de réflexion de conception (hormis `Polygone` qui demandait la gestion d'un tableau dynamique).

Transformations Géométriques :

Les transformations géométriques qui ont été demandées d'être effectuées sont la translation, l'homothétie et la rotation. Avec un peu de réflexion, il a déterminé assez simplement qu'une opération géométrique s'effectuait avant tout sur un point (pouvant être écrit comme un vecteur pour rappel) et non une forme en particulier. De ce fait, l'implémentation de ces transformations se devait être faite dans trois classes : Forme (de manière purement abstraite), Forme Simple et Forme Composée. Pour Forme Simple, il s'agissait dans un premier d'appeler `getPoints()`, permettant de collecter l'ensemble des points d'une Forme Simple et qui était implémenté par Segment, Cercle, Triangle et Polygone, de réaliser ensuite les transformations géométriques sur ces points, et à partir des nouveaux points reçus, de les appliquer aux formes grâce à `setPoints()` qui était aussi implémenté par Segment, Cercle, Triangle et Polygone. La seule difficulté résidait pour Cercle de retrouver le rayon après les transformations. Pour cela, `getPoints()` pour Cercle retournait son centre mais aussi le point du coin bas-gauche et le point haut-droit du carré tel que Cercle était son cercle inscrit, et pour `setPoints()`, à partir des points des angles du carré, d'en faire la norme puis de diviser le résultat par $2\sqrt{2}$, nous donnons à présent le nouveau rayon du cercle. A propos de Forme Composée, les transformations géométriques de cette dernière consistaient en les transformations géométriques de ses composantes, ce qui était assez simple à implémenter finalement. Enfin, pour les opérations mathématiques sur les points, la translation s'agissait d'une simple addition de vecteur, l'homothétie d'une soustraction par un vecteur, d'une multiplication par une constante et de l'addition par le précédent vecteur, et enfin, la rotation d'une soustraction par un vecteur, d'une multiplication par une matrice formée par un angle radian et de l'addition par le précédent vecteur.

Dessin d'une forme :

Le dessin d'une forme, bien qu'effrayant dans un premier temps par le nombre d'éléments différents à gérer, a finalement été pas si compliqué à réaliser. En effet, dans un premier temps, il se devait pour chaque classe finale de nos formes de rajouter une fonction `accept()` permettant l'introduction de nouvelle méthode par ce qu'on appelle le design pattern visiteur, permettant donc à ce que le dessin puisse être très facilement changé dans le futur. Ces méthodes appelaient toutes la même fonction avec le même paramètre demandé, à savoir la forme elle-même. Venait ensuite une série d'étapes bien distinctes. Dans un premier lieu la création du socket permettant la connexion avec le serveur Java. Dans un second temps a lieu l'envoi de la couleur de la forme au serveur Java. Dans un troisième vient la transformation mathématique de chaque point permettant à ce que chaque forme qui devait être dessinée d'être correctement affichée sur l'écran. Enfin, vient l'envoi de chaque forme en `toString()`, et de la demande de les dessiner. Nous obtenons ainsi les protocoles suivants pour la communication :

Message	Action
Couleur <Couleur>	Prend en charge la couleur utile pour le dessin
Segment <Couleur> <Coordonnées>	Crée un segment côté serveur
Cercle <Couleur> <Coordonnées> <Rayon>	Crée un cercle côté serveur
Triangle <Couleur> <Coordonnées>	Crée un triangle côté serveur
Polygone <Couleur> <Coordonnées>	Crée un polygone côté serveur
FormeComposee <Couleur> [	Ignore
]	Ignore
Dessiner	Dessine les formes faites côté java, les colorie de la couleur souhaitée, affiche les dessins, et efface les formes et remet la couleur à défaut

Le serveur Java, démarré de manière indépendant de son côté et opérant un thread permettant l'interception des différentes tentatives de connexion sans perturber le traitement des autres demandes des autres sockets, reçoit la tentative de connexion, crée l'espace pour le socket, et grâce un thread permet de gérer la réception des messages liés à ce socket sans interrompre² le reste des autres processus. A chaque message reçu, celui-ci passe par une chaîne de responsabilité permettant ainsi une plus grande extensibilité du code en cas de rajout de nouvelles formes notamment. Enfin, à chaque message est lié son propre traitement comme décrit précédemment. Le dessin d'une forme se fait grâce à la librairie awt, avec une gestion de l'active rendering permettant de meilleures performances pour l'affichage. A chaque dessin lui est associé sa propre fenêtre.

Sauvegarder/Chargement sur un fichier disque :

Afin de réaliser les deux tâches, il a été nécessaire d'implémenter deux design pattern :

- Pour la sauvegarder, le design pattern Visitor a été utilisées. Pour chaque classe finale, nous avons une fonction accept() tel qu'elle accepte un visiteur, permettant une plus grande extensibilité du programme. Pour chaque fonction du visiteur qui intègre chaque forme, il est simplement demandé le toString() de ces dites formes. Enfin, ce toString() est stocké dans un fichier dans le nom est choisi par l'utilisateur.
- Pour le chargement, la design pattern Chain Of Responsabilité a été mis en place. A partir d'un fichier, on en extrait le texte pour la faire passer dans différents maillons d'une chaîne, qui, si elle détecte quelque chose ayant un lien avec une forme, retourne un pointeur de ladite forme. Un cas particulier est celui de Forme Composée, où est il nécessaire de connaître l'origine du maillon afin de pouvoir stocker dans le Forme Composée l'ensemble des Formes qui la compose.

Fonctionnalités diverses :

Il a été demandé dans ce projet d'implémenter deux fonctionnalités :

- La première, le calcul d'aire. Etant donné que le calcul d'air dépend grandement de la Forme, celle-ci doit être implémenté partout et définit dans les classes finales. Pour le calcul de l'aire d'un Segment, rien de plus simple, il est égal à 0. Pour le calcul de l'aire d'un Cercle, il suffit de calculer $\pi 2r$, r étant le rayon du Cercle. Pour le Triangle, il suffit de calculer le déterminant en fonction de ses points et de le diviser par deux. Pour le Polygone, il suffit de le trianguliser et d'appliquer la même méthode pour le Triangle. Enfin, l'aire d'une Forme Composée est finalement l'aire de l'ensemble des Formes Simples qui le composent.
- La deuxième, la conversion en string. De la même manière que le calcul de l'aire, la fonction doit être implémenté dans l'ensemble des classes et définit dans les classes finales. Pour chaque Forme Simple, il suffit dans un premier temps de retourner le nom de la forme (Segment, Triangle, etc...), puis sa couleur, et enfin les coordonnées de chaque point de manière brut. Ce qui donne quelque chose du genre « Triangle Red 1 5 6 4 -3 9 ». Un cas particulier est celui du cercle, où à la fin est énoncé la longueur de son rayon. Dans le cas des Formes Composées, il suffit dans un premier temps d'enregistrer son nom, puis sa couleur, suivi d'un « [» et d'un saut de ligne. Puis, chaque Forme composant la Forme Composée est écrite en sautant une ligne à chaque fois. Enfin, nous mettons un «] » pour conclure la Forme Composée.

Documentation :

Afin de rendre le code simple à comprendre et à maintenir, chaque objet, membre et méthodes ont été décrites grâce à la notation Doxygen.

Cela inclut donc pour chaque élément une description par tag qui sont les suivantes :

- @brief : indication du comportement global d'un élément
- @param[in] : paramètre d'entrée
- @return : retour d'une fonction
- @details : précision sur les paramètres d'entrées et les possibles exception.